

Golden Brown Haven

Hotel Management System

Project Documentation

This document explains the actual implementation I used in this individual full-stack project. It focuses on the Spring Boot backend, React frontend, MySQL database, JWT security flow, booking logic, room management, AWS S3 image upload support, and dashboard/UI structure found in the repository.

Backend Spring Boot 4, Java 21, Spring Security, Spring Data JPA	Frontend React 19, Vite, Material UI, Axios, React Router
Database MySQL with JPA entities and repository layer	Security BCrypt password hashing, JWT bearer tokens, method-level roles
Media Multipart upload flow with AWS S3 SDK support	

1. Project Overview

Golden Brown Haven is a hotel management web application I built as an individual project. The project has a customer side for browsing rooms and making bookings, and an admin side for managing rooms, users, bookings, and dashboard analytics.

- **Main purpose:** Practice a real full-stack application flow instead of building only static pages.
- **Core problem:** Keep hotel rooms, bookings, and users connected through one backend API and database layer.
- **User value:** Customers can browse available rooms, place bookings, see booking history, and cancel bookings.
- **Admin value:** Admins can view bookings, manage rooms, view users, and use charts to understand room and booking data.
- **Project scope:** This version does not include WebSocket realtime, OTP, email verification, or forgot-password backend flows.

2. System Architecture

The project follows a normal full-stack request flow. The React frontend calls REST endpoints from the Spring Boot API. The backend keeps business logic in services, database access in repositories, and persisted data in MySQL tables generated from JPA entities.

User / Browser

```
-> React + Vite frontend
-> Axios HTTP request
-> Spring Boot REST controller
-> Service layer business logic
-> Spring Data JPA repository
-> MySQL database
-> DTO / Response object back to frontend
```

- **Authentication flow:** Login sends email and password to /auth/login, Spring Security authenticates the user, and JWTUtils creates a signed token.
- **Protected API flow:** The frontend stores the token and sends it using the Authorization bearer header for dashboard actions.
- **Booking flow:** Customer selects a room and dates, backend checks room availability, saves the booking, and returns a generated confirmation code.
- **File upload flow:** Admin room creation/update sends multipart form data. The backend uploads the image with the AWS S3 SDK and stores the returned image URL on the room record.

3. Backend Implementation

Framework and Structure

The backend is a Spring Boot project using Java 21. I separated the code into controllers, services, repositories, entities, DTOs, security classes, and utility classes so each part has a clear responsibility.

Controllers

AuthController handles register and login. RoomController handles room CRUD and availability search. BookingController handles booking creation, lookup, listing, and cancellation. UserController handles user list, profile, and booking history endpoints.

Services and Business Logic

UserService handles registration, login, password encoding, JWT response data, user lookup, and user deletion. RoomService handles room creation, updates, deletion, room lookup, availability filtering, and S3 uploads. BookingService validates date ranges, checks room availability, saves bookings, and generates confirmation codes.

Repositories

The project uses Spring Data JPA repositories for User, Room, and Booking. Custom repository methods support email lookup, booking confirmation lookup, booking history by user, and room availability queries.

4. Database Model

The database layer is based on JPA entities. With `spring.jpa.hibernate.ddl-auto=update`, Hibernate can update the MySQL schema from the entity classes during local development.

- **User:** Stores id, name, email, phone number, hashed password, role, and linked bookings. User implements UserDetails for Spring Security authentication.
- **Room:** Stores id, room type, A/C type, price, capacity, image URL, description, and linked bookings.
- **Booking:** Stores check-in/check-out dates, adults, children, phone number, total guests, confirmation code, linked user, and linked room.
- **Relationships:** User -> Booking is one-to-many. Room -> Booking is one-to-many. Booking -> User and Booking -> Room are many-to-one relations.

```
users
  id, email, name, phone_number, password, role
  one user can have many bookings

rooms
  id, room_type, ac_type, room_price, room_capacity,
  room_photo_url, room_description
  one room can have many bookings

bookings
  id, check_in_date, check_out_date, guests,
  phone_number, booking_confirmation_code,
  user_id, room_id
```

5. Authentication and Security

- **Registration:** New users are saved with ROLE_USER by default when no role is provided. Passwords are encoded with BCrypt before saving.
- **Login:** The backend authenticates email and password through AuthenticationManager and returns a JWT token, role, and user DTO.
- **JWT filter:** JWTAuthFilter reads the Authorization header, extracts the email from the token, loads the user, validates token expiry and username, then sets the Spring Security context.
- **Stateless sessions:** SecurityConfig disables CSRF for API usage and sets session creation policy to STATELESS.
- **Role checks:** Several controller methods use @PreAuthorize for ADMIN and ROLE_USER access, especially booking management, room update/delete, and admin user listing.
- **Sensitive config:** Database passwords, AWS keys, JWT secrets, and email/API credentials should not be committed. Keep real values in local ignored files or environment variables.
- **Current limits:** The current codebase does not implement HttpOnly cookies, OTP, email verification, password reset, or password update flows.

6. Frontend Implementation

The frontend is a Vite React application built with Material UI components, React Router routes, Axios API calls, Framer Motion animation, Recharts dashboards, and responsive dashboard layouts.

- **Routes:** /, /signup, /login, /admin-home, and /customer-home are configured in App.jsx.
- **Authentication handling:** Login stores the token, user id, and user object in localStorage, then redirects users based on ROLE_ADMIN or ROLE_USER.
- **Customer dashboard:** Loads rooms, user profile, my bookings, booking history, room details, available room search, booking creation, and cancellation.
- **Admin dashboard:** Loads bookings, rooms, and users in parallel, supports room add/update/delete, user delete, dashboard summaries, and analytics charts.
- **UI layer:** Material UI provides forms, cards, tables, dialogs, drawers, modals, steppers, tabs, charts, snackbars, and responsive layout controls.
- **Visual polish:** The UI uses dark/light mode toggles, animated cards, loading states, validation messages, and dashboard charts. The landing page uses static showcase data and a local assistant widget, not a backend AI service.

7. Important Features

Room Availability Search

The room search endpoint accepts check-in date, check-out date, room type, and A/C type from the frontend. The backend checks bookings to avoid returning unavailable rooms for the selected date range.

Booking Confirmation

When a booking is saved, the backend generates a random alphanumeric confirmation code. The frontend stores and displays the code so the customer can keep it for check-in.

Admin Analytics

The admin dashboard calculates chart data from loaded room and booking arrays. It includes A/C vs non-A/C room distribution, booking comparisons, revenue-style summaries, occupancy-style calculations, and Recharts visualizations.

Image Upload

Room creation and update support multipart image upload. AwsS3Service sends the file to an S3 bucket and returns the public image URL, which is stored on the Room entity.

8. Setup and Configuration Notes

Local setup needs Java 21, Maven, Node.js, npm, and MySQL. The backend runs on port 8080 and the frontend Vite dev server normally runs on port 5173.

Backend

```
cd backend
mvn spring-boot:run
```

Frontend

```
cd frontend/ui
npm install
npm run dev
```

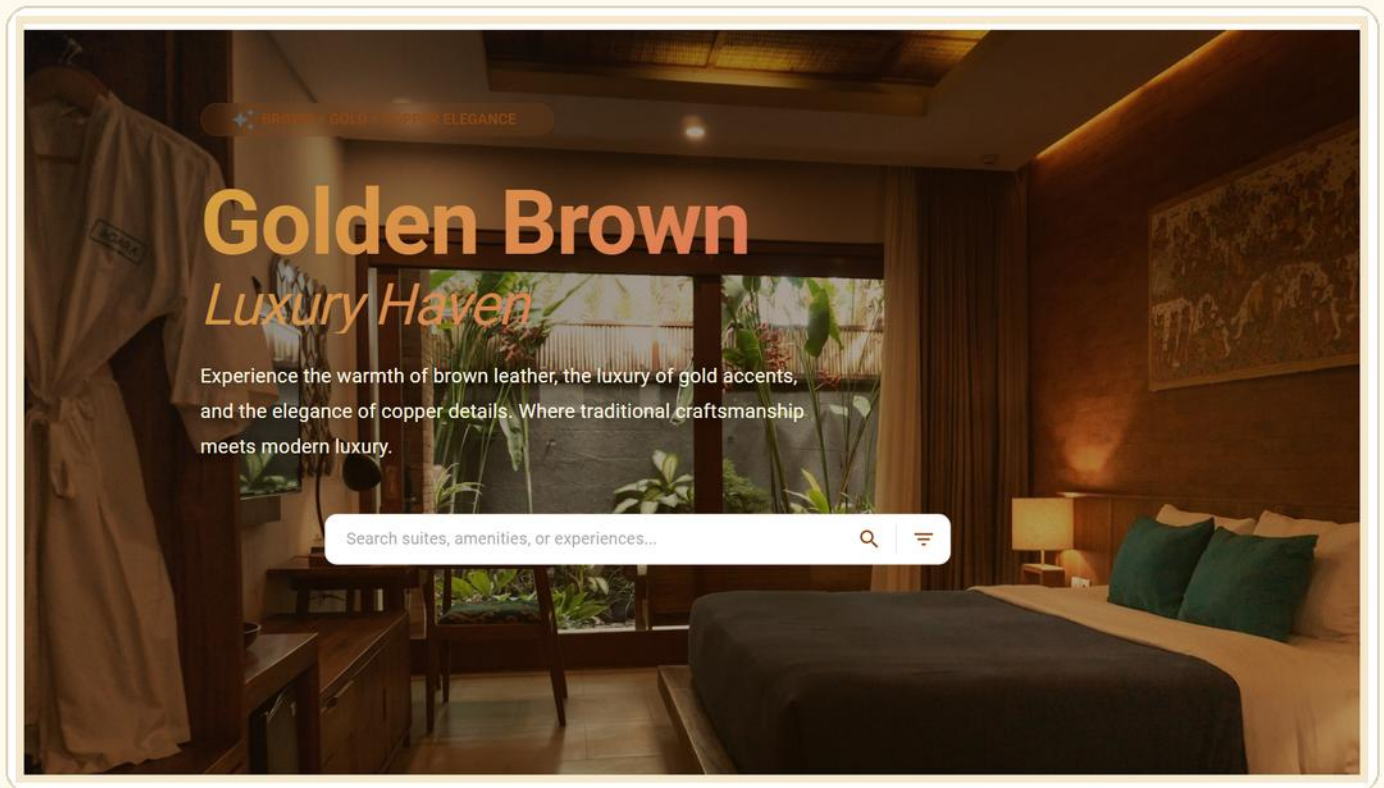
Database

```
Create or allow MySQL database: hotelDb
Configure datasource URL, username, and password locally.
```

- **Config safety:** Do not push real values in `application.properties`, `.env`, or any local configuration file.
- **Safe Git cleanup:** If a sensitive file was already tracked, use `git rm --cached path/to/file`, then add that path to `.gitignore`. This keeps the local file on disk.
- **Future improvements:** Tests, stronger backend route protection, better validation, deployment config, centralized logging, and password recovery can improve the next version.

9. UI Preview

The full page-by-page UI walkthrough is available in docs/ui-screenshots.pdf. I kept this technical PDF focused on implementation details and included the landing page preview below as a quick visual reference.



- **Landing page:** Public hotel brand page with hero image, search bar, auth links, theme toggle, and animated presentation sections.
- **Auth pages:** Multi-step signup and login connect to Spring Boot /auth endpoints.
- **Dashboards:** Customer and admin pages connect to backend APIs for bookings, rooms, users, and charts.